

# VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT  
on

## OPERATING SYSTEMS

Submitted by

**DEEPIKA (1BM25CS436)**

in partial fulfillment for the award of the degree of  
**BACHELOR OF ENGINEERING**  
in  
**COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING**

(Autonomous Institution under VTU)

**BENGALURU-560019**

**Feb-2026 to June-2026**

B. M. S. College of Engineering,  
Bull Temple Road, Bangalore 560019  
(Affiliated To Visvesvaraya Technological University, Belgaum)  
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “**OPERATING SYSTEMS – 23CS4PCOPS**” carried out by **DEEPIKA (1BM25CS436)**, who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026 . The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name  
Assistant Professor  
Department of CSE  
BMSCE, Bengaluru

Dr. Kavitha Sooda  
Professor and Head  
Department of CSE  
BMSCE, Bengaluru

### Index Sheet

| Sl. No. | Experiment Title                                                                                                                                                                                                                                                                                                                           | Page No. |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 1.      | Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.<br>→FCFS<br>→ SJF (pre-emptive & Non-preemptive)<br>→ Priority (pre-emptive & Non-pre-emptive)<br>→Round Robin                                                                                              |          |
| 2.      | Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue. |          |
| 3.      | Write a C program to simulate Real-Time CPU Scheduling algorithms:<br>a) Rate- Monotonic<br>b) Earliest-deadline First<br>Proportional scheduling                                                                                                                                                                                          |          |
| 4.      | Write a C program to simulate:<br>a) Producer-Consumer problem using semaphores.<br>b) Dining-Philosopher's problem                                                                                                                                                                                                                        |          |
| 5.      | Write a C program to simulate:<br>a) Bankers' algorithm for the purpose of deadlock avoidance.<br>b) Deadlock Detection                                                                                                                                                                                                                    |          |
| 6.      | Write a C program to simulate the following contiguous memory allocation techniques.<br>a) Worst-fit<br>b) Best-fit<br>c) First-fit                                                                                                                                                                                                        |          |
| 7.      | Write a C program to simulate page replacement algorithms.<br>a) FIFO<br>b) LRU<br>c) Optimal                                                                                                                                                                                                                                              |          |

### Course Outcomes

|     |                                                                                     |
|-----|-------------------------------------------------------------------------------------|
| C01 | Apply the different concepts and functionalities of Operating System                |
| C02 | Analyse various Operating system strategies and techniques                          |
| C03 | Demonstrate the different functionalities of Operating System.                      |
| C04 | Conduct practical experiments to implement the functionalities of Operating system. |

## Program -1

### Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

→ Priority (pre-emptive & Non-pre-emptive)

→Round Robin (Experiment with different quantum sizes for RR algorithm)

### Code:

=>FCFS:

```
#include<stdio.h>
```

```
void sort(int proc_id[],int at[],int bt[],int n)
```

```
{
    int min=at[0],temp=0;
    for(int i=0;i<n;i++)
    {
        min=at[i];
    }
    for(int j=i;j<n;j++)
    {
        if(at[j]<min)
        {
            temp=at[i];
            at[i]=at[j];
            at[j]=temp;
            temp=bt[j];
            bt[j]=bt[i];
            bt[i]=temp;
            temp=proc_id[i];
            proc_id[i]=proc_id[j];
            proc_id[j]=temp;
        }
    }
}
```

```
void main()
```

```
{
```

```

    int n,c=0;    printf("Enter number of
processes: ");    scanf("%d",&n);    int
proc_id[n],at[n],bt[n],ct[n],tat[n],wt[n];
double
avg_tat=0.0,ttat=0.0,avg_wt=0.0,twt=0.
0;    for(int i=0;i<n;i++)
proc_id[i]=i+1;    printf("Enter arrival
times:\n");    for(int i=0;i<n;i++)
scanf("%d",&at[i]);    printf("Enter
burst times:\n");    for(int i=0;i<n;i++)
scanf("%d",&bt[i]);

    sort(proc_id,at,bt,n);
//completion time
for(int i=0;i<n;i++)
    {        if(c>=at[i])
c+=bt[i];        else
c+=at[i]-ct[i-1]+bt[i];
ct[i]=c;
    }
//turnaround time
for(int i=0;i<n;i++)
tat[i]=ct[i]-at[i];
//waiting time
for(int i=0;i<n;i++)
wt[i]=tat[i]-bt[i];

    printf("FCFS scheduling:\n");    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++)
printf("%d\t%d\t%d\t%d\t%d\t%d\n",proc_id[i],at[i],bt[i],ct[i],tat[i],wt[i]);

    for(int i=0;i<n;i++)
    {
ttat+=tat[i];twt+=wt[i];
    }
    avg_tat=ttat/(double)n;    avg_wt=twt/(double)n;
printf("\nAverage turnaround time:%lfms\n",avg_tat);
printf("\nAverage waiting time:%lfms\n",avg_wt);
}

```

## OUTPUT:

Output:-

Enter number of process:

4

enter arrival time:

4 0 1 5 6

enter Burst time:

2 2 3 4

| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| 1   | 0  | 2  | 2  | 2   | 0  |
| 2   | 1  | 2  | 4  | 3   | 1  |
| 3   | 5  | 3  | 8  | 3   | 0  |
| 4   | 6  | 4  | 12 | 6   | 2  |

average waiting time = 0.75  
average turnaround time = 3.50

Gantt chart:

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| P1 | P2 | P1 | P3 | P4 |    |
| 0  | 2  | 4  | 5  | 8  | 12 |

De 12/12

=> SJF(Non-preemptive):

```
#include<stdio.h>
```

```
int main()
{
    int n,i,j,pos,temp;
    int at[10],bt[10],ct[10],tat[10],wt[10],p[10];

    float avg_tat=0,avg_wt=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);
```

```

printf("Enter Arrival Time:\n");
for(i=0;i<n;i++)
{
    scanf("%d",&at[i]);
    p[i]=i+1;
}
printf("Enter Burst Time:\n");
for(i=0;i<n;i++)
scanf("%d",&bt[i]);

for(i=0;i<n;i++)
{
    pos=i;
    for(j=i+1;j<n;j++)
    {
        if(bt[j] < bt[pos] && at[j] <= at[i])
            pos=j;
    }
    temp=bt[i];
    bt[i]=bt[pos];
    bt[pos]=temp;

    temp=at[i];
    at[i]=at[pos];
    at[pos]=temp;

    temp=p[i];
    p[i]=p[pos];
    p[pos]=temp;
}
ct[0]=at[0]+bt[0];

for(i=1;i<n;i++)
{
    if(ct[i-1] < at[i])
        ct[i]=at[i]+bt[i];
    else
        ct[i]=ct[i-1]+bt[i];
}
for(i=0;i<n;i++)

```



```

{
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];

    avg_tat+=tat[i];
    avg_wt+=wt[i];
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

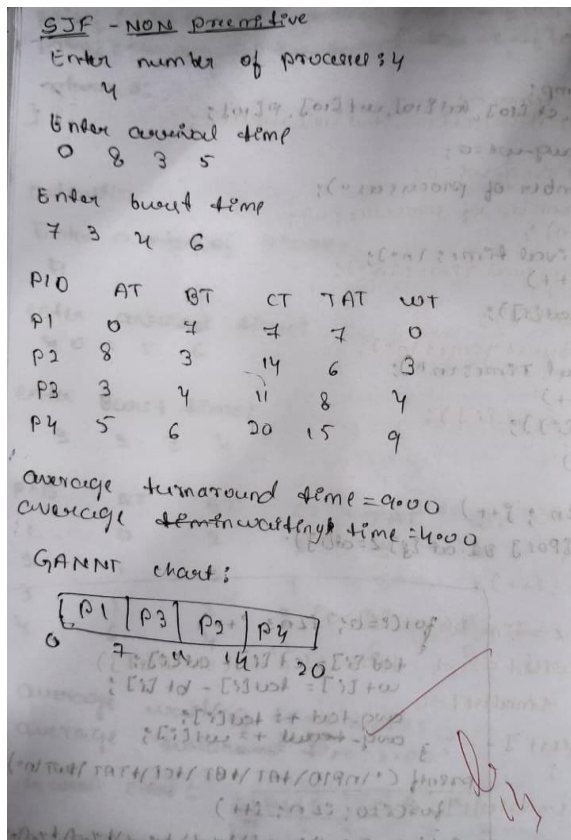
for(i=0;i<n;i++)
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);

printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f",avg_wt/n);

return 0;
}

```

## OUTPUT:



=>SJF (pre-emptive):

```
#include<stdio.h>
```

```
int main()
{
    int n,i,time=0,remain;
    int at[10],bt[10],rt[10],ct[10],tat[10],wt[10];
    int smallest;
    float avg_tat=0,avg_wt=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time:\n");
    for(i=0;i<n;i++)
        scanf("%d",&at[i]);
```

```

printf("Enter Burst Time:\n");
for(i=0;i<n;i++)
{
    scanf("%d",&bt[i]);
    rt[i]=bt[i];
}
remain=n;
while(remain!=0)
{
    smallest=-1;

    for(i=0;i<n;i++)
    {
        if(at[i]<=time && rt[i]>0)
        {
            if(smallest==-1 || rt[i] < rt[smallest])
                smallest=i;
        }
    }
    rt[smallest]--;
    time++;
    if(rt[smallest]==0)
    {
        remain--;
        ct[smallest]=time;
    }
}
for(i=0;i<n;i++)
{
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];

    avg_tat+=tat[i];
    avg_wt+=wt[i];
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(i=0;i<n;i++)
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f",avg_wt/n);

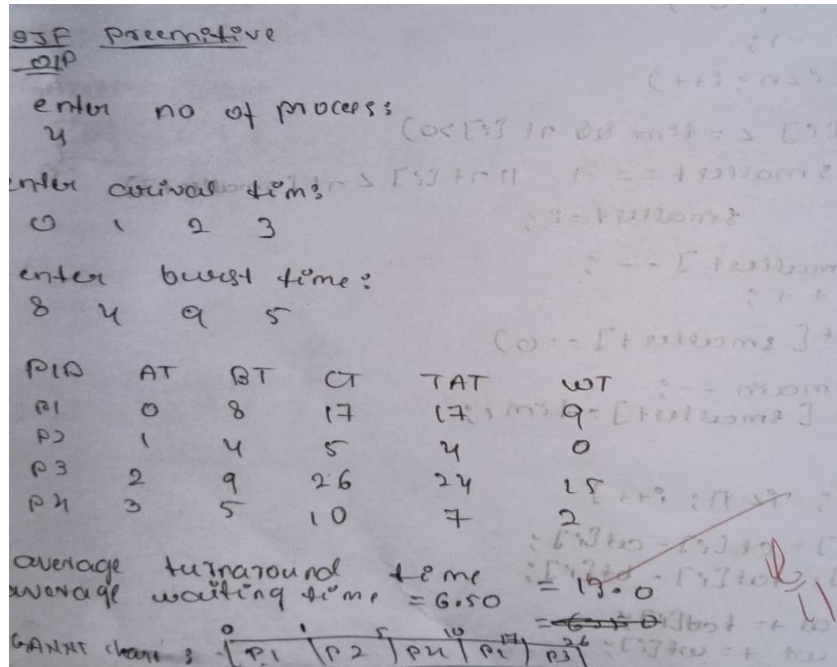
```

```

return 0;
}

```

## OUTPUT:



## Code:

=>Priority(non-pre-emptive):

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int n,i,time=0,completed=0;
```

```
int at[10],bt[10],pr[10],ct[10],tat[10],wt[10];
```

```
int p[10],visited[10]={0};
```

```
float avgwt=0,avgtat=0;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d",&n);
```

```
for(i=0;i<n;i++)
```

```
{
```

```
printf("Enter AT BT Priority for P%d: ",i+1);
```

```

scanf("%d%d%d", &at[i], &bt[i], &pr[i]);
p[i]=i+1;
}
printf("\nGantt Chart:\n");

while(completed<n)
{
    int idx=-1;
    int highest=999;

    for(i=0;i<n;i++)
    {
        if(at[i]<=time && visited[i]==0)
        {
            if(pr[i]<highest)
            {
                highest=pr[i];
                idx=i;
            }
        }
    }
    if(idx!=-1)
    {
        printf(" P%d |", p[idx]);
        time+=bt[idx];
        ct[idx]=time;

        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];

        avgwt+=wt[idx];
        avgtat+=tat[idx];

        visited[idx]=1;
        completed++;
    }
    else
    {
        time++;
    }
}

```

```

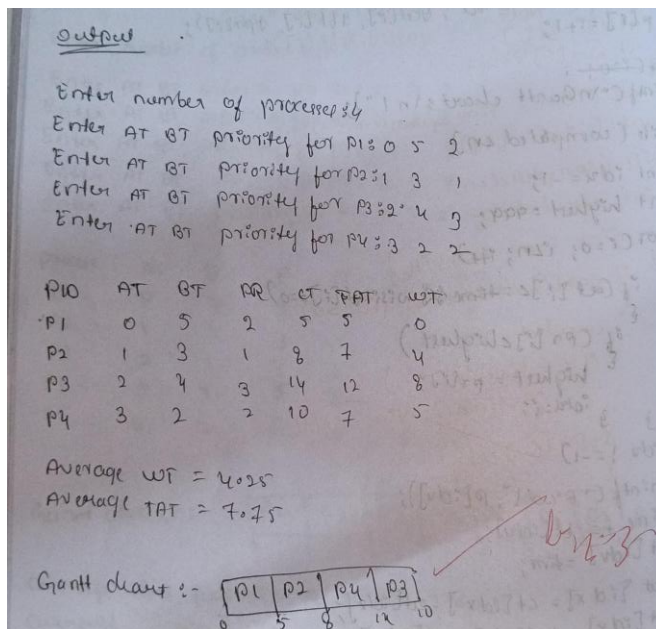
    }
    printf("\n\nP\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    for(i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage WT = %.2f",avgwt/n);
    printf("\nAverage TAT = %.2f",avgtat/n);

    return 0;
}

```

### OUTPUT:



=>Priority(pre-emptive):

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,time=0,remain;
```

```
    int at[10],bt[10],pr[10];
```

```

int rt[10],ct[10],tat[10],wt[10];
int max,loc;

printf("Enter number of processes: ");
scanf("%d",&n);

for(i=0;i<n;i++)
{
    printf("Enter AT BT Priority for P%d: ",i+1);
    scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
    rt[i]=bt[i];
}
remain=n;

while(remain!=0)
{
    max=-1;
    loc=-1;

    for(i=0;i<n;i++)
    {
        if(at[i]<=time && rt[i]>0)
        {
            if(pr[i]>max)
            {
                max=pr[i];
                loc=i;
            }
        }
    }
    if(loc!=-1)
    {
        rt[loc]--;
        time++;

        if(rt[loc]==0)
        {
            ct[loc]=time;
            remain--;
        }
    }
    else

```

```

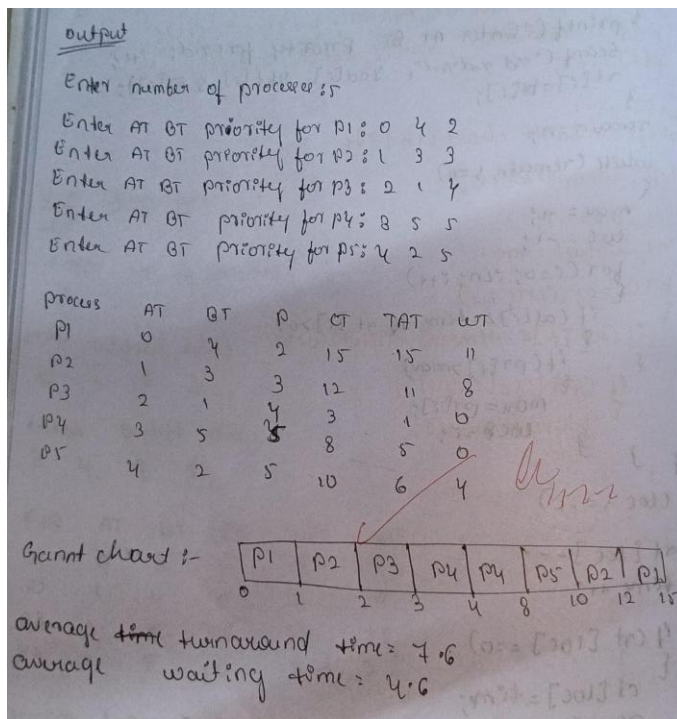
        time++;
    }
    printf("\nProcess\tAT\tBT\tP\tCT\tTAT\tWT\n");

    for(i=0;i<n;i++)
    {
        tat[i]=ct[i]-at[i];
        wt[i]=tat[i]-bt[i];

        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    }
    return 0;
}

```

### OUTPUT:





**=>Round Robin:**

```
#include <stdio.h>

int main()
{
    int n, tq;
    int at[10], bt[10], rt[10];
    int ct[10], tat[10], wt[10], rt_time[10];
    int time = 0, completed = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(int i=0;i<n;i++)
    {
        printf("Enter AT and BT for P%d: ",i+1);
        scanf("%d%d",&at[i],&bt[i]);
        rt[i] = bt[i];
        rt_time[i] = -1;
    }

    printf("Enter Time Quantum: ");
    scanf("%d",&tq);

    while(completed < n)
    {
        int executed = 0;

        for(int i=0;i<n;i++)
        {
            if(at[i] <= time && rt[i] > 0)
            {
                executed = 1;

                if(rt_time[i] == -1)
                    rt_time[i] = time - at[i];

                if(rt[i] > tq)
                {
                    time += tq;
                }
            }
        }
    }
}
```

```

        rt[i] -= tq;
    }
    else
    {
        time += rt[i];
        rt[i] = 0;
        ct[i] = time;
        completed++;
    }
}

if(executed == 0)
    time++;
}
float avgwt = 0, avgtat = 0;

printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(int i=0;i<n;i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    avgwt += wt[i];
    avgtat += tat[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt_time[i]);
}
printf("\nAverage WT = %.2f",avgwt/n);
printf("\nAverage TAT = %.2f",avgtat/n);

return 0;
}

```

## OUTPUT:

output

Enter number of processes: 6

Enter AT & BT for P1: 0 4

Enter AT & BT for P2: 1 5

Enter AT & BT for P3: 2 2

Enter AT & BT for P4: 3 1

Enter AT & BT for P5: 4 6

Enter AT & BT for P6: 6 3

Enter time quantum: 2

| PID | AT | BT | CT | TAT | WT | RT |
|-----|----|----|----|-----|----|----|
| P1  | 0  | 4  | 8  | 8   | 4  | 0  |
| P2  | 1  | 5  | 18 | 17  | 12 | 1  |
| P3  | 2  | 2  | 6  | 4   | 2  | 2  |
| P4  | 3  | 1  | 9  | 6   | 5  | 5  |
| P5  | 4  | 6  | 21 | 17  | 4  | 5  |
| P6  | 6  | 3  | 19 | 13  | 10 | 7  |

Average waiting time = 7.33

Average turnaround time = 10.83

antt chart:

|    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|
| P1 | P2 | P3 | P1 | P4 | P5 | P2 | P6 | P5 | P2 | P6 | P5 |    |
| 0  | 2  | 4  | 6  | 8  | 9  | 11 | 13 | 15 | 17 | 18 | 19 | 21 |

## Program-2:

### Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

### Code:

=>Multi-level Queue:

```

#include <stdio.h>
#define MAX 20

struct Process {
    int pid, at, bt, rt;
    int ct, tat, wt;
    int qno;
};

int main() {

    int n;
    struct Process p[MAX];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter AT BT QueueNo(1-RR,2-FCFS):\n");

    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].qno);
        p[i].rt = p[i].bt;
    }

    int time = 0, completed = 0;
    int tq = 2;

    while (completed < n) {

        int executed = 0;

```

```

for (int i = 0; i < n; i++) {

    if (p[i].qno == 1 &&
        p[i].rt > 0 &&
        p[i].at <= time) {

        executed = 1;

        int run = (p[i].rt > tq) ? tq : p[i].rt;

        time += run;
        p[i].rt -= run;

        if (p[i].rt == 0) {
            p[i].ct = time;
            completed++;
        }
    }
}

if (executed) continue;

int idx = -1;
int earliest = 99999;

for (int i = 0; i < n; i++) {
    if (p[i].qno == 2 &&
        p[i].rt > 0 &&
        p[i].at <= time) {

        if (p[i].at < earliest) {

```

```

        earliest = p[i].at;
        idx = i;
    }
}
}
if (idx != -1) {

    executed = 1;

    while (p[idx].rt > 0) {
        int q1_ready = 0;

        for (int j = 0; j < n; j++) {
            if (p[j].qno == 1 &&
                p[j].rt > 0 &&
                p[j].at <= time) {
                q1_ready = 1;
                break;
            }
        }
        if (q1_ready)
            break;
        time++;
        p[idx].rt--;
    }
    if (p[idx].rt == 0) {
        p[idx].ct = time;
        completed++;
    }
}
if (!executed)

```

```

        time++;
    }
    float avg_wt = 0, avg_tat = 0;

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

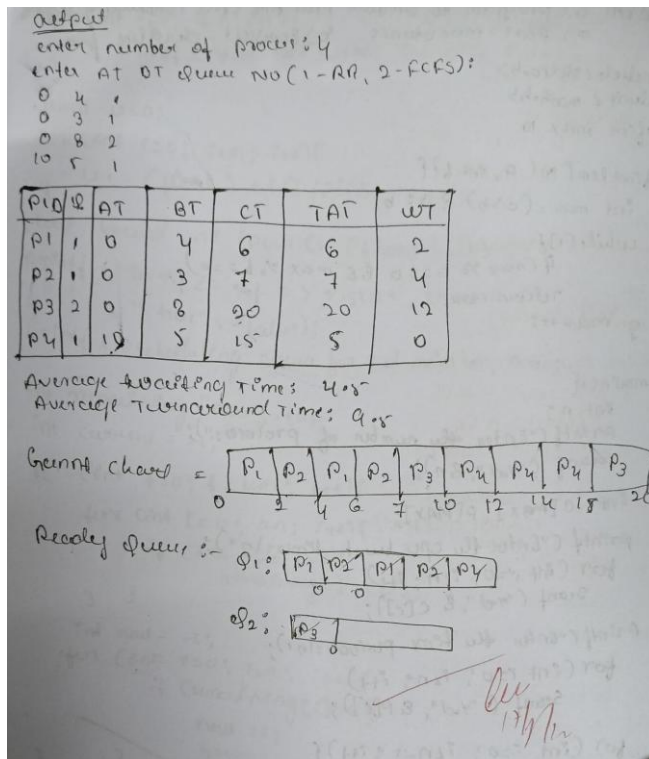
    for (int i = 0; i < n; i++)
    {
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,
            p[i].at,
            p[i].bt,
            p[i].ct,
            p[i].tat,
            p[i].wt);

        avg_wt += p[i].wt;
        avg_tat += p[i].tat;
    }
    printf("\nAvg WT = %.2f", avg_wt / n);
    printf("\nAvg TAT = %.2f\n", avg_tat / n);
    return 0;
}

```

**OUTPUT:**



### Program-3:

#### Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- Rate- Monotonic
- Earliest-deadline First
- Proportional scheduling

#### Code:

=>Rate- Monotonic:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int gcd(int a, int b) {
```

```
    if (b == 0)
```



```

        return a;
    return gcd(b, a % b);
}
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
int find_lcm(int arr[], int n) {
    int res = arr[0];
    for (int i = 1; i < n; i++) {
        res = lcm(res, arr[i]);
    }
    return res;
}
int main()
{
    int n, i, time;
    int burst[MAX], period[MAX], remaining[MAX];

    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the CPU burst times:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &burst[i]);
    printf("Enter the time periods:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &period[i]);

    // Initialize remaining time
    for (i = 0; i < n; i++)
        remaining[i] = 0;
    int hyper = find_lcm(period, n);

```

```

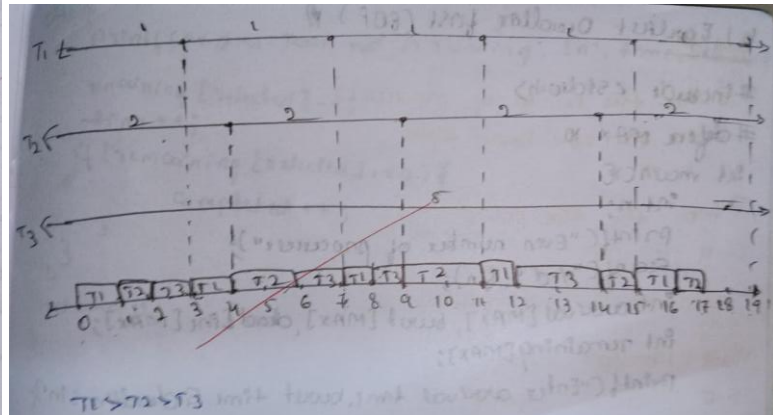
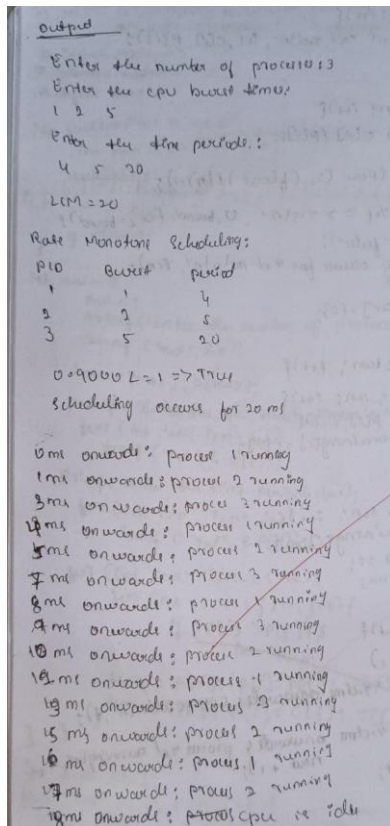
printf("\nLCM = %d\n", hyper);

printf("\nRate Monotonic Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
}
printf("\nScheduling occurs for %d ms\n\n", hyper);
for (time = 0; time < hyper; time++) {

    // Release tasks at period
    for (i = 0; i < n; i++) {
        if (time % period[i] == 0) {
            remaining[i] = burst[i];
        }
    }
    int selected = -1;
    int min_period = 9999;
    // Select task with smallest period (highest priority)
    for (i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            if (period[i] < min_period) {
                min_period = period[i];
                selected = i;
            }
        }
    }
    if (selected != -1) {
        printf("%d ms onwards : Process %d running\n", time, selected + 1);
        remaining[selected]--;
    } else {

```

**OUTPUT:**



```
int gcd(int a, int b) {
    if (b == 0)
```

```

        return a;
    return gcd(b, a % b);
}
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
int find_lcm(int arr[], int n) {
    int res = arr[0];
    for (int i = 1; i < n; i++) {
        res = lcm(res, arr[i]);
    }
    return res;
}
int main() {
    int n, i, time;
    int burst[MAX], period[MAX], remaining[MAX];

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the CPU burst times:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &burst[i]);

    printf("Enter the time periods:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &period[i]);

    // Initialize remaining time
    for (i = 0; i < n; i++)
        remaining[i] = 0;

```

```

int hyper = find_lcm(period, n);
printf("\nLCM = %d\n", hyper);

printf("\nRate Monotonic Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
}
printf("\nScheduling occurs for %d ms\n\n", hyper);

for (time = 0; time < hyper; time++)
{
    // Release tasks at period
    for (i = 0; i < n; i++) {
        if (time % period[i] == 0) {
            remaining[i] = burst[i];
        }
    }
    int selected = -1;
    int min_period = 9999;

    // Select task with smallest period (highest priority)
    for (i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            if (period[i] < min_period) {
                min_period = period[i];
                selected = i;
            }
        }
    }
    if (selected != -1) {

```

```

printf("%d ms onwards : Process %d running\n", time, selected + 1);
remaining[selected]--;
} else {
printf("%d ms onwards : CPU is idle\n", time);
}
}
return 0;
}

```

### OUTPUT:

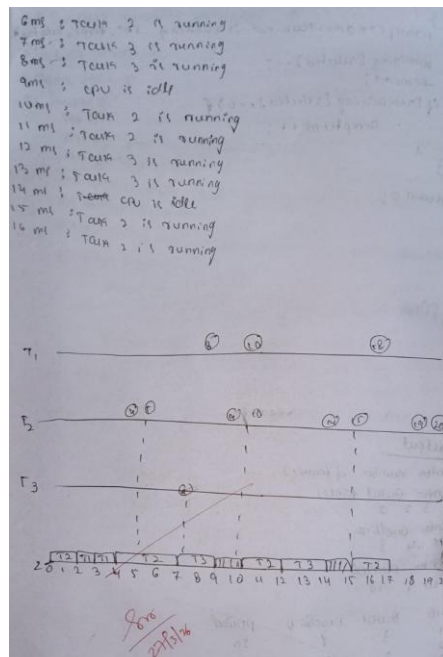
output

Enter number of tasks: 3  
Enter Burst times:  
3 2 2  
Enter Deadlines:  
7 4 8  
Enter periods:  
20 5 10

| PID | Burst | Deadline | period |
|-----|-------|----------|--------|
| 1   | 3     | 7        | 20     |
| 2   | 2     | 4        | 5      |
| 3   | 2     | 8        | 10     |

scheduling occurs for 20 ms

0ms : Task 2 is Running  
1ms : Task 2 is Running  
2ms : Task 1 is Running  
3ms : Task 1 is Running  
4ms : Task 1 is Running  
5ms : Task 1 is Running



### =>Proportional Share Scheduling:

```

#include<stdio.h>
#include <stdlib.h>
#include <time.h>

```

```

typedef struct

```

```

{

```

```

char name[5]; int tickets; }

```

```

Process;
int main() {
int n, total_tickets = 0; float total_T = 0.0;
printf("Enter the number of Processes: ");
scanf("%d", &n);
Process p[n];
srand(time(NULL));
for (int i = 0; i < n; i++)
{ printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T += p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d",&m);
for (int i = 0; i < m; i++)
{
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets)
{
winner_index = j; break;
}
}
}
}

```

```

printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winning_index].name);
}
for (int i = 0; i < n; i++)
{
printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
((p[i].tickets / total_T) * 100));
}
return 0;
}

```

### OUTPUT:

```

output
Enter the number of processes: 3
process 1:
tickets: 10
process 2:
tickets: 20
process 3:
tickets: 30
--- proportional share scheduling ---
Enter the time period for scheduling: r
Tickets picked: 44, winner: p3
Tickets picked: 26, winner: p2
Tickets picked: 33, winner: p2
Tickets picked: 55, winner: p3
Tickets picked: 57, winner: p3
The process: p1 gets 16.67% of processor time
The process: p2 gets 33.33% of processor time
The process: p3 gets 50.00% of processor time

```

### Program-4:

#### Question:

Write a C program to simulate:

- Producer-Consumer problem using semaphores.
- Dining-Philosopher's problem

#### Code:

=> Producer-Consumer problem using semaphores.



```

#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#include<semaphore.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty; // counts empty slots
sem_t full; // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion
void* producer(void* arg) {
int item, i = 0;
while (1) {
item = i++; // produce an item (here just incrementing number)
sem_wait(&empty); // wait for empty slot
pthread_mutex_lock(&mutex); // enter critical section
buffer[in] = item;
printf("Produced: %d at buffer[%d]\n", item, in);
in = (in + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex); // leave critical section
sem_post(&full); // signal that new item is available
sleep(1); // simulate time to produce
}
}
void* consumer(void* arg)
{
int item;
while (1)
{
sem_wait(&full); // wait for filled slot

```

```

pthread_mutex_lock(&mutex); // enter critical section
item = buffer[out];
printf("Consumed: %d from buffer[%d]\n", item, out);
out = (out + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex); // leave critical section
sem_post(&empty); // signal that a slot is free
sleep(2); // simulate time to consume
}
}
int main()
{
pthread_t prod, cons;
sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0); pthread_mutex_init(&mutex, NULL);
pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);
pthread_join(prod, NULL);
pthread_join(cons, NULL);
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);
return 0;

}

```

## OUTPUT:

OUTPUT

```

produced: 0 at buffer [0]
consumed: 0 from buffer [0]
produced: 1 at buffer [1]
consumed: 1 from buffer [1]
produced: 2 from at buffer [2]
consumed: 2 from buffer [2]
produced: 3 at buffer [3]
consumed: 3 from buffer [3]
.
.
.

```

### =>Dining-Philosopher:

#### Code:

```

#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#define N 5 // Number of philosophers
pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];
void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id; // left fork index
    int right = (id + 1) % N; // right fork index

```

```

while (1)
{
printf("Philosopher %d is thinking.\n", id);
sleep(1); // Thinking
// To avoid deadlock, philosophers with even id pick left then right,
// odd id pick right then left.
if (id % 2 == 0)
{
// Pick up left fork
pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);
// Pick up right fork

pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right); }
else
{ // Pick up right fork
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right); // Pick up left
fork pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);
}
// Eating
printf("Philosopher %d is eating.\n", id);
sleep(2); // Put down forks
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}
return NULL;
}

```

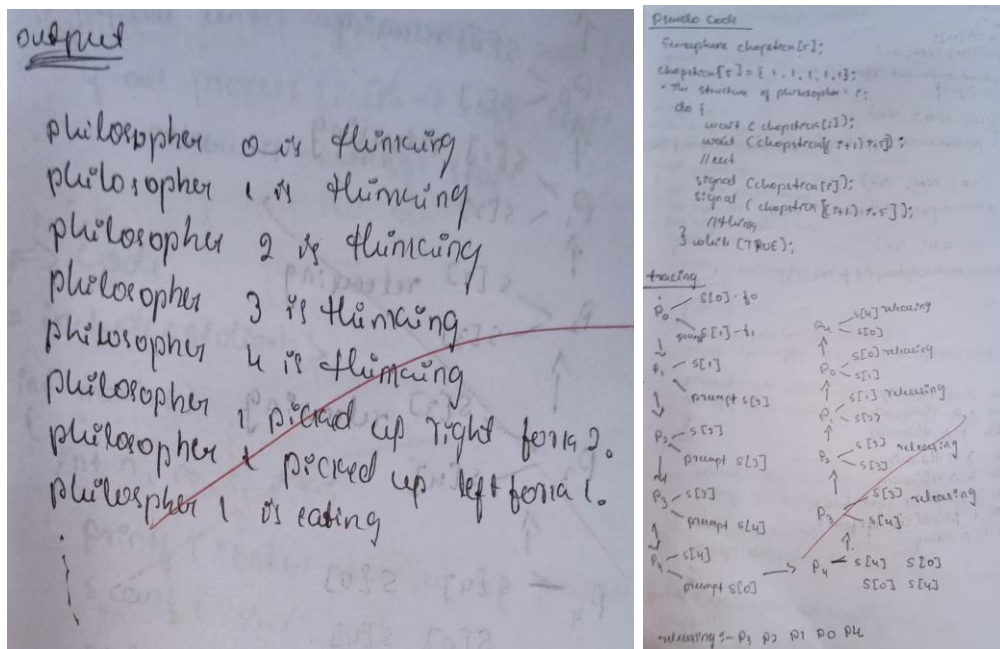
```

int main() {
    int i;
    int ids[N]; // Initialize forks (mutexes)
    for (i = 0; i < N; i++)
    { pthread_mutex_init(&forks[i], NULL);
      ids[i] = i;
    }
    // Create philosopher threads
    for (i = 0; i < N; i++)

    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    } // Join threads (will never happen here, but good practice)
    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL); }
    // Destroy mutexes (unreachable here)
    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }
    return 0;
}

```

**OUTPUT:**



## Program-5:

### Question:

Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

→Banker's Algorithm

→Deadlock Detection

### Code

⇒Banker's Algorithm:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m, i, j, k;
```

```

printf("Enter number of processes: ");
scanf("%d", &n);

printf("Enter number of resources: ");
scanf("%d", &m);

int alloc[n][m], max[n][m], need[n][m];
int avail[m];
printf("Enter Allocation Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        scanf("%d", &alloc[i][j]);
    }
}
printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}
printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
{
    scanf("%d", &avail[i]);
}
// Calculate Need Matrix
for(i = 0; i < n; i++)

```

```

{
    for(j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}
int finish[n], safeSeq[n];
for(i = 0; i < n; i++)
{
    finish[i] = 0;
}
int count = 0;
while(count < n)
{
    int found = 0;
    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int flag = 1;
            for(j = 0; j < m; j++)
            {
                if(need[i][j] > avail[j])
                {
                    flag = 0;
                    break;
                }
            }
            if(flag)
            {
                for(k = 0; k < m; k++)

```



```

        {
            avail[k] += alloc[i][k];
        }
        safeSeq[count] = i;
        count++;
        finish[i] = 1;
        found = 1;
    }
}
}
if(found == 0)
{
    printf("\nSystem is not in safe state.\n");
    return 0;
}
}
printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");
for(i = 0; i < n; i++)
{
    printf("P%d", safeSeq[i]);
    if(i != n - 1)
        printf(" -> ");
}
printf("\n");
return 0;
}

```

**OUTPUT:**

Output

Enter number of processes: 5  
Enter number of resource: 3  
Enter Allocation Matrix:  
0 1 0  
2 0 0  
3 0 2  
2 1 1  
0 0 2  
Enter Maximum Demand matrix:  
7 5 3  
3 2 2  
4 0 2  
2 2 2  
4 3 3

Enter available Resources:  
3 3 2

system is in a safe state.  
safe sequence is: P1 → P3 → P4 → P0 → P2

| need  |
|-------|
| 7 4 3 |
| 1 2 2 |
| 6 0 0 |
| 0 4 1 |
| 4 3 1 |

=>Deadlock Detection:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m, i, j, k;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the number of resources: ");
```

```

scanf("%d", &m);

int alloc[n][m], request[n][m], avail[m];
int finish[n], safeSeq[n];

printf("Enter the allocation matrix:\n");
for(i = 0; i < n; i++)
{
    printf("Process %d: ", i);

    for(j = 0; j < m; j++)
    {
        scanf("%d", &alloc[i][j]);
    }
}
printf("Enter the request matrix:\n");
for(i = 0; i < n; i++)
{
    printf("Process %d: ", i);

    for(j = 0; j < m; j++)
    {
        scanf("%d", &request[i][j]);
    }
}
printf("Enter the available resources: ");
for(i = 0; i < m; i++)
{
    scanf("%d", &avail[i]);
}
// Initialize finish array

```

```

for(i = 0; i < n; i++)
{
    finish[i] = 0;
}
int count = 0;

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int flag = 1;
            for(j = 0; j < m; j++)
            {
                if(request[i][j] > avail[j])
                {
                    flag = 0;
                    break;
                }
            }
            if(flag)
            {
                for(k = 0; k < m; k++)
                {
                    avail[k] += alloc[i][k];
                }
                safeSeq[count] = i;
                count++;
            }
        }
    }
}

```

```

        finish[i] = 1;
        found = 1;
    }
}
}
if(found == 0)
{
    break;
}
}
if(count == n)
{
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSeq[i]);

        if(i != n - 1)
            printf(" ");
    }
    printf("\n");
}
else
{
    printf("\nDeadlock detected.\nProcesses in deadlock are: ");
    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {

```

```

        printf("P%d ", i);
    }
}
printf("\n");
}
return 0;
}

```

## OUTPUT:

output

Enter the number of process: 5  
Enter the number of resource: 3  
Enter the allocation matrix:

|            |   |   |   |
|------------|---|---|---|
| process 0: | 0 | 1 | 0 |
| process 1: | 2 | 0 | 0 |
| process 2: | 3 | 0 | 3 |
| process 3: | 2 | 1 | 1 |
| process 4: | 0 | 0 | 2 |

Enter the request matrix:

|            |   |   |   |
|------------|---|---|---|
| process 0: | 0 | 0 | 0 |
| process 1: | 0 | 0 | 2 |
| process 2: | 0 | 0 | 0 |
| process 3: | 1 | 0 | 0 |
| process 4: | 0 | 0 | 2 |

Enter the available resource: 0 0 0  
System is in safe state  
Safe sequence P1, P0, P2, P3, P4, P1

Process 0:  $work = 0, 0, 0$   
 $request[0] = 0, 0, 0$   
 $P_0: F[0] = \text{false}$   
 $Request \leq work$   
 $0, 0, 0 \leq 0, 0, 0$  (true)  
 $work = work + allocation$   
 $work = 0, 0, 0 + 0, 1, 0 = 0, 1, 0$   
 $F[0] = \text{true}$   
 $P_1: F[1] = \text{false}$   
 $Request \leq work$   
 $2, 0, 2 \leq 0, 1, 0$  (false)  
 $P_2: F[2] = \text{false}$   
 $0, 0, 0 \leq 0, 1, 0$  (true)  
 $0, 1, 0 + 3, 0, 3$   
 $work = 3, 1, 3$   
 $F[2] = \text{true}$   
 $P_3: F[3] = \text{false}$   
 $1, 0, 0 \leq 3, 1, 3$   
 $2, 1, 1 + 3, 1, 3$   
 $work = 5, 2, 4$   
 $P_4: F[4] = \text{false}$   
 $0, 0, 2 \leq 5, 2, 4$   
 $work = work + allocation$   
 $0, 0, 2 + 0, 0, 2 = 0, 0, 4$   
 $work = 5, 2, 6$   
 $P_0: F[0] = \text{false}$   
 $2, 0, 2 \leq 5, 2, 6$   
 $work = 2, 0, 0 + 5, 2, 6$   
 $work = 7, 2, 6$   
Safe sequence:  $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$   
 $\therefore$  No deadlock.

## **Program-6:**

### **Question:**

Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

### **code:**

**=>Worst-fit ,Best-fit, First-fit:**

```
#include <stdio.h>

int main() {
    int m, n, i, j;
    int blockSize[20], processSize[20];
    int allocation[20];
    int choice;

    printf("CONTIGUOUS MEMORY ALLOCATION\n");
    printf("1. First Fit\n");
    printf("2. Best Fit\n");
    printf("3. Worst Fit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    printf("Enter size of each block:\n");
    for(i = 0; i < m; i++) {
```

```

        scanf("%d", &blockSize[i]);
    }
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter size of each process:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }
    for(i = 0; i < n; i++) {
        allocation[i] = -1;
    }
    // FIRST FIT
    if(choice == 1) {
        for(i = 0; i < n; i++) {
            for(j = 0; j < m; j++) {
                if(blockSize[j] >= processSize[i]) {
                    allocation[i] = j;
                    blockSize[j] -= processSize[i];
                    break;
                }
            }
        }
        printf("\nFIRST FIT ALLOCATION\n");
    }
    // BEST FIT
    else if(choice == 2) {
        for(i = 0; i < n; i++) {
            int bestIdx = -1;
            for(j = 0; j < m; j++)
            {

```



```

        if(blockSize[j] >= processSize[i]) {
            if(bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                bestIdx = j;
            }
        }
    }
    if(bestIdx != -1) {
        allocation[i] = bestIdx;
        blockSize[bestIdx] -= processSize[i];
    }
}
printf("\nBEST FIT ALLOCATION\n");
}

// WORST FIT
else if(choice == 3) {
    for(i = 0; i < n; i++) {
        int worstIdx = -1;

        for(j = 0; j < m; j++) {
            if(blockSize[j] >= processSize[i]) {
                if(worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
                    worstIdx = j;
                }
            }
        }
        if(worstIdx != -1) {
            allocation[i] = worstIdx;
            blockSize[worstIdx] -= processSize[i];
        }
    }
    printf("\nWORST FIT ALLOCATION\n");
}

```

```

    }
else {
    printf("Invalid Choice\n");
    return 0;
}
printf("Process No\tProcess Size\tBlock No\n");

for(i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, processSize[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
return 0;
}

```

## OUTPUT:

Output

Enter Size of 5 memory blocks  
100 500 200 300 600

Enter number of processes: 4

Enter Size of 4 processes:  
212 417 112 426

--- First Fit ---

| Process No | Process Size | Block No      |
|------------|--------------|---------------|
| 1          | 212          | 2             |
| 2          | 417          | 5             |
| 3          | 112          | 2             |
| 4          | 426          | Not allocated |

--- Best Fit ---

| Process No | Process Size | Block No |
|------------|--------------|----------|
| 1          | 212          | 4        |
| 2          | 417          | 2        |
| 3          | 112          | 3        |
| 4          | 426          | 5        |

--- Worst Fit ---

| Process No | Process Size | Block No      |
|------------|--------------|---------------|
| 1          | 212          | 5             |
| 2          | 417          | 2             |
| 3          | 112          | 5             |
| 4          | 426          | Not allocated |

## **Program-7:**

### **Question:**

Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

**code:**

**=>FIFO,LRU,Optimal:**

```
#include <stdio.h>

void printFrames(int frames[], int num_frames) {
    for(int i = 0; i < num_frames; i++) {
        if(frames[i] == -1)
            printf("- ");
        else
            printf("%d ", frames[i]);
    }
}

// ----- FIFO -----

void FIFO(int pages[], int num_pages, int num_frames) {

    int frames[num_frames];
    int page_faults = 0;
    int next = 0;

    for(int i = 0; i < num_frames; i++)
        frames[i] = -1;

    printf("\n===== FIFO Page Replacement =====\n");
```

```

for(int i = 0; i < num_pages; i++) {
    int found = 0;
    for(int j = 0; j < num_frames; j++) {
        if(frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }
    if(found == 0) {
        frames[next] = pages[i];
        next = (next + 1) % num_frames;
        page_faults++;
    }
    printf("Page %d -> ", pages[i]);
    printFrames(frames, num_frames);

    if(found == 0)
        printf(" Page Fault");
    printf("\n");
}
printf("Total Page Faults = %d\n", page_faults);
}

// ----- OPTIMAL -----
void OPTIMAL(int pages[], int num_pages, int num_frames) {

    int frames[num_frames];
    int page_faults = 0;

    for(int i = 0; i < num_frames; i++)
        frames[i] = -1;

```

```

printf("\n===== Optimal Page Replacement =====\n");
for(int i = 0; i < num_pages; i++) {

    int found = 0;
    for(int j = 0; j < num_frames; j++) {
        if(frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }
    if(found == 0) {

        int empty = -1;

        for(int j = 0; j < num_frames; j++) {
            if(frames[j] == -1) {
                empty = j;
                break;
            }
        }
        if(empty != -1) {
            frames[empty] = pages[i];
        }
        else
        {
            int farthest = -1;
            int replace_index = -1;

            for(int j = 0; j < num_frames; j++) {
                int k;

```

```

        for(k = i + 1; k < num_pages; k++) {
            if(frames[j] == pages[k])
                break;
        }
        if(k == num_pages) {
            replace_index = j;
            break;
        }
        if(k > farthest) {
            farthest = k;
            replace_index = j;
        }
    }
    frames[replace_index] = pages[i];
}
page_faults++;
}
printf("Page %d -> ", pages[i]);
printFrames(frames, num_frames);

if(found == 0)
    printf(" Page Fault");
printf("\n");
}
printf("Total Page Faults = %d\n", page_faults);
}
// ----- LRU -----
void LRU(int pages[], int num_pages, int num_frames) {

    int frames[num_frames];
    int time[num_frames];

```

```

int page_faults = 0;

for(int i = 0; i < num_frames; i++) {
    frames[i] = -1;
    time[i] = -1;
}
printf("\n==== LRU Page Replacement =====\n");
for(int i = 0; i < num_pages; i++) {

    int found = 0;
    for(int j = 0; j < num_frames; j++) {

        if(frames[j] == pages[i]) {
            found = 1;
            time[j] = i;
            break;
        }
    }
    if(found == 0) {

        int empty = -1;
        for(int j = 0; j < num_frames; j++) {
            if(frames[j] == -1) {
                empty = j;
                break;
            }
        }
        if(empty != -1) {
            frames[empty] = pages[i];
            time[empty] = i;
        }
    }
}

```

```

else {
    int lru_index = 0;

    for(int j = 1; j < num_frames; j++) {
        if(time[j] < time[lru_index])
            lru_index = j;
    }
    frames[lru_index] = pages[i];
    time[lru_index] = i;
}
page_faults++;
}
printf("Page %d -> ", pages[i]);
printFrames(frames, num_frames);

if(found == 0)
    printf(" Page Fault");

    printf("\n");
}
printf("Total Page Faults = %d\n", page_faults);
}

// ----- MAIN -----
int main() {
    int num_frames, num_pages;

    printf("Enter number of frames: ");
    scanf("%d", &num_frames);

    printf("Enter number of pages: ");

```



```

scanf("%d", &num_pages);

int pages[num_pages];
printf("Enter reference string:\n");
for(int i = 0; i < num_pages; i++) {
    scanf("%d", &pages[i]);
}

FIFO(pages, num_pages, num_frames);
OPTIMAL(pages, num_pages, num_frames);
LRU(pages, num_pages, num_frames);

return 0;
}

```

## OUTPUT:

output

Enter number of frames: 3  
Enter number of pages: 11  
Enter reference string:  
5 0 1 0 2 3 0 2 4 3 3 0 0 0 1 2 7 0 1

-- FIFO page Replacement --

page 5 → 5 -- page fault  
page 0 → [ 5 0 ]  
page 1 → [ 5 0 1 ]  
page 0 → [ 5 0 1 ]  
page 2 → [ 2 0 1 ]  
page 3 → [ 2 3 1 ]  
page 0 → [ 2 3 0 ]  
page 2 → [ 2 3 0 ]  
page 4 → [ 4 3 0 ]  
page 3 → [ 4 3 0 ]  
page 3 → [ 4 3 0 ]  
page 0 → [ 4 3 0 ]  
page 0 → [ 4 3 0 ]  
page 2 → [ 4 2 0 ]  
page 1 → [ 4 2 1 ]  
page 2 → [ 4 2 1 ]  
page 7 → [ 7 2 1 ]

page 0 → [ 1 0 7 ]  
page 1 → [ 1 0 7 ]  
page 1 → [ 1 0 7 ]  
page 0 → [ 1 0 7 ]

-- Optimal page Replacement --

page 5 → [ 5 - - ]  
page 0 → [ 5 0 - ]  
page 1 → [ 5 0 1 ]  
page 0 → [ 5 0 1 ]  
page 2 → [ 2 0 1 ]  
page 3 → [ 2 0 3 ]  
page 0 → [ 2 0 3 ]  
page 2 → [ 2 0 3 ]  
page 4 → [ 2 4 3 ]  
page 3 → [ 2 4 3 ]  
page 3 → [ 2 4 3 ]  
page 0 → [ 2 4 0 ]  
page 0 → [ 2 4 0 ]  
page 2 → [ 2 4 0 ]  
page 1 → [ 2 1 0 ]  
page 2 → [ 2 1 0 ]  
page 7 → [ 7 1 0 ]  
page 0 → [ 7 1 0 ]  
page 1 → [ 7 1 0 ]  
page 1 → [ 7 1 0 ]  
page 0 → [ 7 1 0 ]

-- LRU page Replacement --

page 5 → [ 5 - - ]  
page 0 → [ 5 0 - ]  
page 1 → [ 5 0 1 ]  
page 0 → [ 5 0 1 ]  
page 2 → [ 2 0 1 ]  
page 3 → [ 2 0 3 ]  
page 0 → [ 2 0 3 ]  
page 2 → [ 2 0 3 ]  
page 4 → [ 2 0 4 ]  
page 3 → [ 2 3 4 ]  
page 0 → [ 2 3 0 ]  
page 0 → [ 2 3 0 ]  
page 2 → [ 2 3 0 ]  
page 1 → [ 2 1 0 ]  
page 2 → [ 2 1 0 ]  
page 7 → [ 7 1 7 ]

page 0 → [ 1 0 7 ]  
page 1 → [ 1 0 7 ]  
page 1 → [ 1 0 7 ]  
page 0 → [ 1 0 7 ]